

2026-05-07-p2-f23-cline-browser-tool-design

Phase 2 / Feature 23 — Cline Browser Tool

Date: 2026-05-07 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 2 port (codex / aider / **cline** / cursor patterns)

Programme position: F23 is the **third** Phase 2 feature (F21 Codex Approval Modes + F22 Aider Git Auto-Commit shipped before it). T01 (bootstrap) appends an F23 evidence section to docs/improvements/07_phase_2_evidence.md (created in F21-T01, extended by F22-T01); T10 (close-out) records F23's runtime evidence beneath F22's.

1. Goal

Ship a real, end-to-end **6-tool browser-automation suite** for the HelixCode CLI agent, modelled verbatim on **cline**'s Puppeteer-based browser tool surface (`cli_agents/cline/`), so that an LLM driver — and a human via the `/browser` slash command — can drive a real headless Chromium subprocess against a real URL, observe the real DOM, click real elements, type into real inputs, capture real PNG screenshots to disk, and tear the subprocess down cleanly. The feature lands six new tools registered through the F21-extended `Tool` interface (each with the correct `RequiresApproval()` level), plus a `/browser` slash command for status / one-shot navigation / explicit close, plus a `BrowserManager` that owns the single chromium session per Helixcode session via an atomic-pointer with lazy lifecycle (Q1=A — chromedp; Q2=A — six tools; Q3=A — headless default with `HELIXCODE_BROWSER_HEADED=true` opt-in; Q4=A — screenshots written to `tmpdir`, tool result returns the file path; Q5=A — `/browser` slash + 6 tools as primary surface, NO cobra subcommand).

Three concrete user surfaces ship together:

1. `internal/tools/browser/` **package additions** — F23 EXTENDS the existing `internal/tools/browser/` package (which already carries chromedp infrastructure: `Controller`, `ActionExecutor`, `ScreenshotCapture`, `ConsoleMonitor`, `ChromeDiscovery`, `ScreenshotAnnotator`, `ElementSelector` — ~3K lines from earlier work). F23 ADDS a thinner cline-style session façade: `BrowserSession` (encapsulates a `chromedp context.Context` + cancel func + a per-session `tmpdir` for screenshots + a screenshot counter + the discovered chromium binary path), `BrowserManager` (atomic pointer to the current `*BrowserSession` with `EnsureSession(ctx) (*BrowserSession, error)` lazy-create on first `navigate`, `RequireSession() (*BrowserSession, error)` for tools that need an active session, `CloseSession() error` invoked by `browser_close` and `process-exit`), `BrowserOptions` (read at session-create time: headless flag from env + viewport defaults), `Snapshot` (returned from `browser_snapshot` — page URL, page title, content-mode marker, content body capped at 64 KB), `ScreenshotResult` (returned from `browser_screenshot` — absolute file path, file size in bytes, PNG width/height read from the chromedp result). Sentinel errors (`ErrNoActiveSession`, `ErrChromiumNotFound`, `ErrNavigationTimeout`, `ErrSelectorNotFound`, `ErrScreenshotTooLarge`). Env-var constants (`EnvVarHeadedMode`).

2. **Six new Tool-interface implementations** — `browser_navigate`, `browser_snapshot`, `browser_click`, `browser_type`, `browser_screenshot`, `browser_close`. Each in its own file under `internal/tools/browser/` (`navigate_tool.go`, `snapshot_tool.go`, `click_type_tools.go`, `screenshot_tool.go`, `close_tool.go`); each implements the F21-extended `tools.Tool` interface (`Name()` / `Description()` / `Schema()` / `Validate()` / `Execute()` / `Category()` / `RequiresApproval()`). The 6 tools take NO `browser_id` parameter — the `BrowserManager` is the single source of truth for “which session is current” (cline pattern). They register themselves with the `tools.ToolRegistry` via a single `RegisterAll(reg, mgr)` helper. Per-tool `RequiresApproval()` (Q2=A consequence) is mapped per §3.6 of this spec: `browser_navigate` → `LevelEdit` (mutates session state, may visit untrusted URLs), `browser_snapshot` → `LevelReadOnly` (pure read of page DOM), `browser_click` → `LevelEdit` (mutates page state, may trigger destructive forms), `browser_type` → `LevelEdit` (mutates page state, may submit credentials), `browser_screenshot` → `LevelReadOnly` (pure capture of pixels), `browser_close` → `LevelEdit` (mutates session state — terminates a subprocess that may have unsaved page state). NO tool is `LevelRun` (the chromium subprocess is owned by `chromedp`, not exposed as a shell tool); NO tool is `LevelAll` (no recursion).
3. **/browser slash command** (Q5=A) — `internal/commands/browser_command.go`. Three subcommands: `/browser status` (active-session yes/no + chromium path + screenshot tempdir + headed/headless mode), `/browser navigate <url>` (one-shot convenience: ensures a session, navigates, returns the resulting URL + title; equivalent to invoking the `browser_navigate` tool but bypasses LLM tool-selection), `/browser close` (calls `manager.CloseSession()` which cancels the `chromedp` context and tears down the chromium subprocess + cleans the tempdir). NO cobra subcommand. The slash is the human-ergonomic surface; the 6 tools are the LLM-ergonomic surface; both go through the same `BrowserManager`.

The single largest bluff vector for F23 is “browser navigated but never actually loaded the URL” — a tool returns success because `chromedp.Run` did not error, but the page is still on `about:blank`, and a snapshot would (silently) return empty HTML. §5.2 enumerates five such patterns and pins each with positive runtime evidence: `navigate-then-snapshot` MUST return HTML containing a fixture string from a real local `httptest.Server`; `click-then-snapshot` MUST show DOM mutation; screenshot files MUST exist AND be > 1 KB AND begin with the PNG magic bytes (89 50 4E 47 0D 0A 1A 0A); `browser_close` MUST cause a subsequent `browser_snapshot` to fail with `ErrNoActiveSession` (positive evidence the session was actually torn down, not just marked-closed); and `headed-mode` `HELIXCODE_BROWSER_HEADED=true` MUST be readable from the running session’s `BrowserOptions`. CONST-042 (no secret leak): full page contents are NEVER logged at INFO level — only the URL, the snapshot byte length, and the screenshot file path. CONST-043: F23 emits zero `git push` commands; T10’s mirror push to four remotes is ATTEMPTED only with explicit user authorisation per push.

Anti-bluff hot zone (loud): a tool returns a success result but chromium isn’t actually running (the `chromedp` context was created but the `chromedp.Run(navigate)` call timed out and the error was swallowed by a `defer recover()`); a snapshot returns empty HTML and a “successful” result because the implementation reads `chromedp.OuterHTML` against `body` BEFORE `chromedp.WaitReady("body")` resolved; a click “succeeded” because `chromedp`’s selector match returned `nil` error even though the selector matched zero elements (`chromedp` v0.15 changed selector-not-found semantics — must use `chromedp.NodeVisible` + a post-click sleep + a snapshot-comparison to assert real change); a screenshot file exists at the returned path but is 0 bytes (`chromedp`’s

FullScreenshot returned nil, nil and the implementation os.WriteFile-d the nil byte slice); browser_close returned success but pgrep chrome still shows the subprocess running because the chromedp cancel func wasn't called or was called on the wrong context. Each of these maps to a unit + integration + Challenge phase per §5.2.

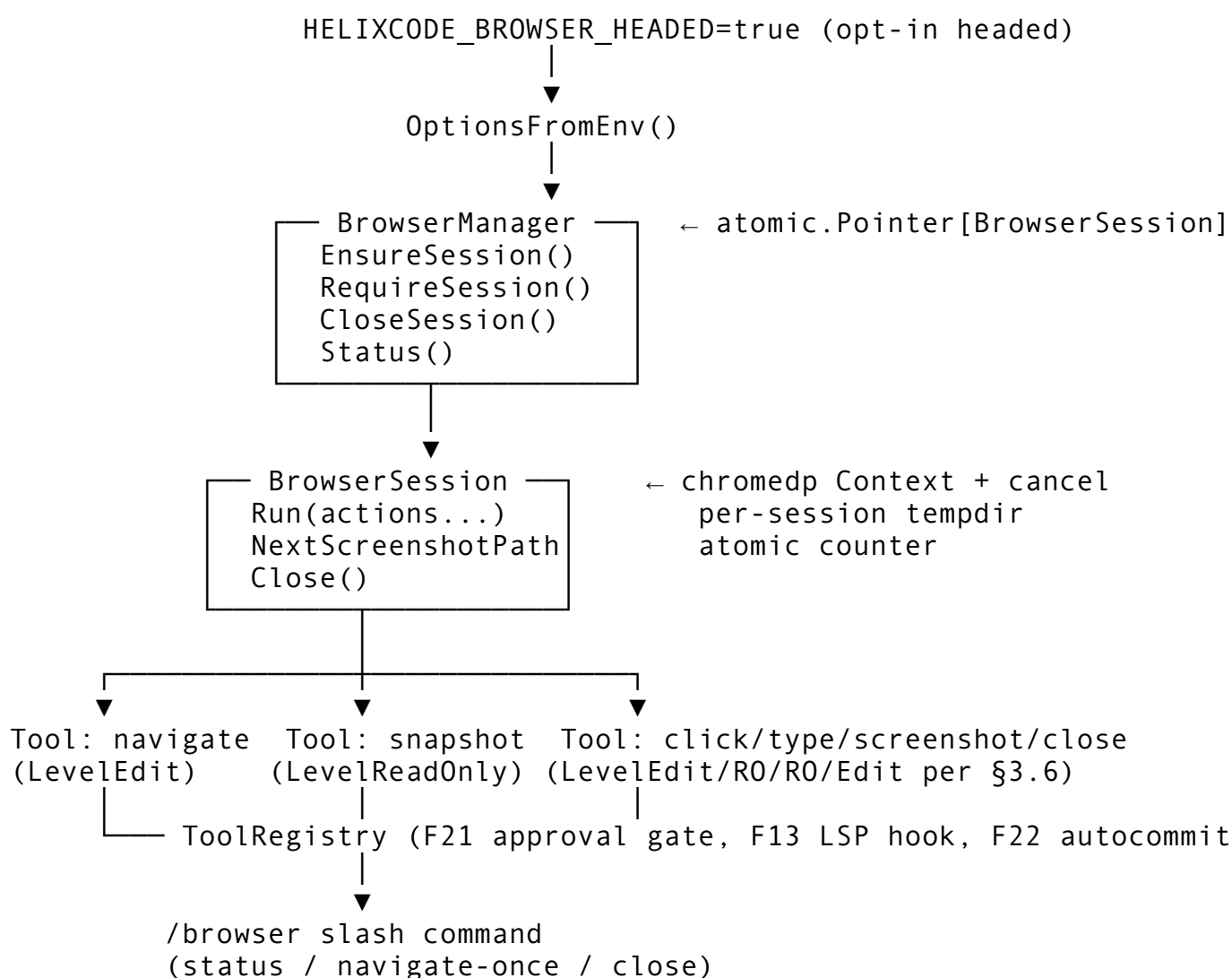
2. Architecture

The package layout under helix_code/internal/tools/browser/ keeps the existing chromedp infrastructure (Controller, ActionExecutor, etc.) intact and ADDS a thin cline-style session façade on top:

- BrowserSession (session.go, NEW) — encapsulates one chromium subprocess. Fields: ctx context.Context (the chromedp Context returned by chromedp.NewContext + chromedp.NewExecAllocator chain), cancel context.CancelFunc (cancels ctx AND tears down the chromium subprocess), screenshotDir string (per-session tmpdir under \$XDG_DATA_HOME/helixcode/browser/screenshots/<session-id>; created at session-create), screenshotCount atomic.Uint64 (next screenshot number; written as <n>.png in the session dir), chromiumPath string (the discovered chromium binary path; recorded for /browser status), headed bool (recorded for /browser status; true when HELIXCODE_BROWSER_HEADED=true was set at session-create), createdAt time.Time (wall-clock when the session started), log *zap.Logger. Methods: Run(ctx context.Context, actions ...chromedp.Action) error (the single chromedp entry point — wraps chromedp.Run(s.ctx, actions...); rejects with ErrNoActiveSession if s.ctx == nil), NextScreenshotPath() string (atomic-increment + format <dir>/<n>.png), Close() error (calls s.cancel() once via sync.Once, removes screenshotDir, returns).
- BrowserManager (manager.go, NEW) — atomic pointer to the single current *BrowserSession. Fields: current atomic.Pointer[BrowserSession], screenshotRoot string (parent of all per-session dirs; usually \$XDG_DATA_HOME/helixcode/browser/screenshots), discovery ChromeDiscovery (the existing internal/tools/browser/discovery.go interface), log *zap.Logger, mu sync.Mutex (guards EnsureSession / CloseSession against races; the atomic pointer makes the read path lock-free). Methods: EnsureSession(ctx context.Context) (*BrowserSession, error) (loads current; if non-nil, returns it; otherwise acquires mu, double-checks, discovers chromium via discovery.Discover, creates the chromedp Allocator + Context, registers cancel + sub-cancel, creates the tmpdir, stores in current, returns), RequireSession() (*BrowserSession, error) (loads current; if nil → ErrNoActiveSession; otherwise returns), CloseSession() error (acquires mu, swaps current to nil, calls s.Close() on the previous session, returns), Status() ManagerStatus (active yes/no + chromium path + tmpdir + headed mode + createdAt). The atomic-pointer pattern ensures concurrent tool calls share one session without the lock-on-read tax; the mutex serialises lifecycle transitions only.
- BrowserOptions (options.go, NEW) — read at EnsureSession time. Fields: Headless bool (default true; overridden by HELIXCODE_BROWSER_HEADED=true), ViewportWidth int (default 1280), ViewportHeight int (default 720), NavigateTimeout time.Duration (default 30 s — covers slow-loading SPAs), ClickWaitDuration time.Duration (default 500 ms — short post-click settle for nav/JS to fire), ScreenshotMaxBytes int64 (default 5 MB — hard cap on full-page screenshots; larger pages get viewport-only

fallback). Read once via `OptionsFromEnv()` at session-create; never re-read mid-session.

- **Snapshot + ScreenshotResult** (types.go, NEW; sit alongside the existing *.go files) — value types returned by `browser_snapshot / browser_screenshot`.
`Snapshot`{URL string; Title string; Mode string; Content string; Truncated bool} (Mode ∈ {html, text} — the user picks via the tool's mode arg; Content is capped at `len(Content) ≤ 64 KB` and Truncated is set when capping happened — positive evidence that capping was deliberate).
`ScreenshotResult`{Path string; Bytes int64; Width int; Height int} — Path is absolute; Bytes > 1024 and PNG-magic invariant are validated by the tool itself before return.
- **Six Tool implementations** — each is a small struct holding `mgr *BrowserManager + opts *BrowserOptions`. `Execute` is the only non-trivial method per tool; the rest are thin (Schema, Description, Validate, Name, Category=CategoryBrowser, RequiresApproval per §3.6).



Why a session-singleton (atomic-pointer) and NOT a session-per-tool-call: chromium spawn cost is ~500 ms; a session-per-call would impose that on every snapshot. The cline pattern is “one session per task, reused across tools”; we follow that. Multi-tab is explicitly out of scope (§8) — the single-tab model matches how an LLM drives a browser (read URL → snapshot → click → snapshot → screenshot → close).

Why a per-session screenshot tempdir (not a single global dir): prevents cross-session filename collisions, and `Close()` can `os.RemoveAll(screenshotDir)` cleanly without racing with another session.

Why screenshot file path return (Q4=A) and NOT base64 inline: a 1 MB PNG base64-encoded blows the tool-result JSON to ~1.4 MB; LLM context windows can't sustain that pattern. The path-return pattern lets downstream tools read the file directly (e.g. the LLM client can attach the file as a multimodal image in the next turn). Tests assert the file exists AND has PNG magic bytes — base64 vs path is a design choice; bytes-on-disk is the runtime evidence.

Why headless-default + env opt-in (Q3=A): headless is the only sensible default for a coding-agent CLI (no display in CI / SSH / containers). `HELIXCODE_BROWSER_HEADED=true` is the documented escape hatch for human-debugging sessions where the developer wants to watch the browser; we don't expose a per-tool `headed:bool` because per-tool toggling would mean tearing down and respawning chromium each time, which is the opposite of the session-singleton design.

Why slash + 6 tools (Q5=A) and NOT cobra: cobra would force a process restart to attach to a new chromium session; the slash + tool surface is purely runtime. The slash gives humans a one-keystroke debug interface (`/browser status`, `/browser close`) without invoking the LLM; the 6 tools give the LLM a granular grip without invoking the human.

Why we do NOT replicate the existing BrowserTools/BrowserLaunchTool-style API: the existing API takes a `browser_id` per call (multi-browser model) and exposes `browser_launch` separately from `browser_navigate`. The cline pattern is: `navigate` creates the session implicitly. F23 lands the cline pattern as a NEW surface; the existing `BrowserLaunchTool/BrowserNavigateTool/BrowserScreenshotTool/BrowserCloseTool` in `internal/tools/browser_tools.go` are NOT removed (would be a backwards-incompatible change for any caller currently using them) — they coexist and are documented as the lower-level multi-browser API; F23's six tools are the recommended primary surface. Spec §3 documents both surfaces; tests exercise only F23's surface.

3. Components

3.1 New files

- `helix_code/internal/tools/browser/types.go` — `Snapshot`, `ScreenshotResult`, `ManagerStatus` value types; sentinel errors; `EnvVarHeadedMode` constant.
- `helix_code/internal/tools/browser/types_test.go`.
- `helix_code/internal/tools/browser/options.go` — `BrowserOptions` + `OptionsFromEnv()`.
- `helix_code/internal/tools/browser/options_test.go`.
- `helix_code/internal/tools/browser/session.go` — `BrowserSession` struct + `Run` + `NextScreenshotPath` + `Close` (`sync.Once-guarded`).
- `helix_code/internal/tools/browser/session_test.go` — exercises `Run` + `NextScreenshotPath` against a stub session built via `chromedp.NewContext` over an in-process headless instance (skip-OK if chromium binary not on PATH).
- `helix_code/internal/tools/browser/manager.go` — `BrowserManager` struct + `EnsureSession` / `RequireSession` / `CloseSession` / `Status`.

- `helix_code/internal/tools/browser/manager_test.go` — atomic-pointer concurrency, double-create idempotency, close-then-require returns `ErrNoActiveSession`.
- `helix_code/internal/tools/browser/navigate_tool.go` — `BrowserNavigateToolV2` (V2 to disambiguate from existing `BrowserNavigateTool` in `internal/tools/browser_tools.go`); registers as Name `browser_navigate` if the existing one is replaced at registration time, OR as a new name `browser_navigate_v2` if coexistence is needed (T09 picks the resolution).
- `helix_code/internal/tools/browser/navigate_tool_test.go`.
- `helix_code/internal/tools/browser/snapshot_tool.go` — `BrowserSnapshotTool`.
- `helix_code/internal/tools/browser/snapshot_tool_test.go`.
- `helix_code/internal/tools/browser/click_type_tools.go` — `BrowserClickTool` + `BrowserTypeTool` together (small, related).
- `helix_code/internal/tools/browser/click_type_tools_test.go`.
- `helix_code/internal/tools/browser/screenshot_tool.go` — `BrowserScreenshotToolV2` (PNG-magic + size verification baked into `Execute`).
- `helix_code/internal/tools/browser/screenshot_tool_test.go`.
- `helix_code/internal/tools/browser/close_tool.go` — `BrowserCloseToolV2`.
- `helix_code/internal/tools/browser/close_tool_test.go`.
- `helix_code/internal/tools/browser/register.go` — `RegisterAll(reg *tools.ToolRegistry, mgr *BrowserManager)` error — single entry point that constructs all six tools and calls `reg.RegisterTool` for each.
- `helix_code/internal/commands/browser_command.go` — `BrowserCommand` slash struct; subcommands `status` / `navigate <url>` / `close`.
- `helix_code/internal/commands/browser_command_test.go`.
- `helix_code/tests/integration/browser_test.go` — `//go:build integration`; gated on chromium availability via `chromedp` discovery; spawns a real `httptest.Server` serving fixture HTML; exercises `navigate` → `snapshot` → `click` → `snapshot` (assert mutation) → `type` → `screenshot` (PNG-magic) → `close` (assert subsequent require fails).
- `helix_code/tests/integration/cmd/p2f23_challenge/main.go` — Challenge harness.
- `challenges/p2-f23-cline-browser-tool/CHALLENGE.md` + `challenges/p2-f23-cline-browser-tool/run.sh`.

3.2 Modified files

- `helix_code/cmd/cli/main.go` — three additions adjacent to F22 wiring: (1) `construct mgr := browser.NewBrowserManager(browser.NewDefaultChromeDiscovery(), c.logger)`; (2) `call browser.RegisterAll(c.toolRegistry, mgr)`; (3) `register commands.NewBrowserCommand(mgr) slash`. NO removal of existing `BrowserLaunchTool`/etc.
- `helix_code/internal/commands/registry.go` — no schema change; one new `Register(...)` call site for `/browser`.
- `helix_code/go.mod` — **zero new external deps** (`chromedp v0.15.1` + `cdproto v0.0.0-20260405000525-47a8ff65b46a` are already direct deps from earlier browser work). T10's verification step asserts `git diff go.mod` and `git diff go.sum` are no-op.

3.3 Types

```
// internal/tools/browser/types.go (NEW; sits alongside existing
types in browser/*.go)

// Snapshot is the result of a browser_snapshot tool call.
type Snapshot struct {
    URL      string `json:"url"`
    Title    string `json:"title"`
    Mode     string `json:"mode"`           // "html" | "text"
    Content  string `json:"content"`         // capped at
                        MaxSnapshotBytes
    Truncated bool   `json:"truncated"`       // true when Content
                        was capped
}

// ScreenshotResult is the result of a browser_screenshot tool
call.
type ScreenshotResult struct {
    Path     string `json:"path"`
    Bytes    int64  `json:"bytes"`
    Width    int    `json:"width"`
    Height   int    `json:"height"`
}

// ManagerStatus is the result of /browser status /
BrowserManager.Status().
type ManagerStatus struct {
    Active        bool   `json:"active"`
    ChromiumPath  string `json:"chromium_path,omitempty"`
    ScreenshotDir string `json:"screenshot_dir,omitempty"`
    Headed        bool   `json:"headed"`
    CreatedAt     time.Time `json:"created_at,omitempty"`
}

const (
    // MaxSnapshotBytes caps Snapshot.Content so a 50 MB SPA HTML
    doesn't
    // blow the tool-result JSON. Truncation sets
    Snapshot.Truncated = true.
    MaxSnapshotBytes = 64 * 1024

    // MaxScreenshotBytes is the hard cap before
    BrowserScreenshotTool falls
    // back to viewport-only capture instead of full-page.
    MaxScreenshotBytes int64 = 5 * 1024 * 1024
)
```

```

// EnvVarHeadedMode is the canonical env var checked by
// OptionsFromEnv at
// EnsureSession time. The literal string "true" (case-
// insensitive) enables
// headed mode; everything else (including unset) means headless.
// Tests pin
// this byte-for-byte.
const EnvVarHeadedMode = "HELIXCODE_BROWSER_HEADED"

var (
    ErrNoActiveSession      = errors.New("browser: no active
        session (call browser_navigate first)")
    ErrChromiumNotFound     = errors.New("browser: chromium binary
        not found in PATH")
    ErrNavigationTimeout    =
        errors.New("browser: navigation timed out")
    ErrSelectorNotFound     =
        errors.New("browser: selector matched zero elements")
    ErrScreenshotTooLarge  = errors.New("browser: screenshot
        exceeds MaxScreenshotBytes")
)

// internal/tools/browser/options.go (NEW)

type BrowserOptions struct {
    Headless          bool
    ViewportWidth     int
    ViewportHeight    int
    NavigateTimeout   time.Duration
    ClickWaitDuration time.Duration
    ScreenshotMaxBytes int64
}

func OptionsFromEnv() BrowserOptions {
    headed := strings.EqualFold(os.Getenv(EnvVarHeadedMode),
        "true")
    return BrowserOptions{
        Headless:          !headed,
        ViewportWidth:     1280,
        ViewportHeight:    720,
        NavigateTimeout:   30 * time.Second,
        ClickWaitDuration: 500 * time.Millisecond,
        ScreenshotMaxBytes: MaxScreenshotBytes,
    }
}

// internal/tools/browser/session.go (NEW)

type BrowserSession struct {

```



```

    ctx          context.Context
    cancel        context.CancelFunc
    screenshotDir string
    screenshotCount atomic.Uint64
    chromiumPath  string
    headed        bool
    createdAt     time.Time
    closeOnce     sync.Once
    log           *zap.Logger
}

func (s *BrowserSession) Run(ctx context.Context,
    actions ...chromedp.Action) error
func (s *BrowserSession) NextScreenshotPath() string
func (s *BrowserSession) Close() error

// internal/tools/browser/manager.go (NEW)

type BrowserManager struct {
    current      atomic.Pointer[BrowserSession]
    screenshotRoot string
    discovery     ChromeDiscovery // existing interface in
                  internal/tools/browser/discovery.go
    log          *zap.Logger
    mu           sync.Mutex
}

func NewBrowserManager(d ChromeDiscovery, log *zap.Logger)
    *BrowserManager
func (m *BrowserManager) EnsureSession(ctx context.Context)
    (*BrowserSession, error)
func (m *BrowserManager) RequireSession() (*BrowserSession,
    error)
func (m *BrowserManager) CloseSession() error
func (m *BrowserManager) Status() ManagerStatus

```

3.4 Tool surface (six tools)

Tool name	RequiresApproval()	Args	Result type	Notes
browser_navigate	LevelEdit	url string (required)	{url, title} map	Lazy-creates body ready; title.
browser_snapshot	LevelReadOnly	mode string (default "html"; allowed "html"/"text")	Snapshot	RequireSessi chromedp. (mode=html) chromedp. (mode=text);

Tool name	RequiresApproval()	Args	Result type	Notes
browser_click	LevelEdit	selector string (required)	{clicked: true, url, title} map	RequireSession, chromedp. ClickWait, Selector-not-ErrSelect
browser_type	LevelEdit	selector string, text string (both required)	{typed: true} map	RequireSession, chromedp. chromedp.
browser_screenshot	LevelReadOnly	full_page bool (default false)	ScreenshotResult	RequireSession, chromedp. 90) (full_page, chromedp. (false) → write s.NextScreenshot > 1024 + PNG
browser_close	LevelEdit	(none)	{closed: true} map	manager.Close, closing a close success).

(Note: per F21, LevelReadOnly bypasses the approval gate entirely; LevelEdit is gated behind ModeAutoEdit or higher. This means snapshot and screenshot are always allowed; navigate/click/type/close require the user to be in a non-ModeSuggest mode OR to confirm at the prompt. This is intentional: pure observation never asks; mutating-the-page or terminating-the-process always asks unless the user has opted into auto-edit.)

3.5 /browser slash command

browser_command.go:

```

type BrowserCommand struct { mgr *BrowserManager }

func NewBrowserCommand(mgr *BrowserManager) *BrowserCommand

func (c *BrowserCommand) Name() string { return "browser" }
func (c *BrowserCommand) Description() string { return "Show browser status, one-shot navigate, or close the session." }
func (c *BrowserCommand) Usage() string { return "/browser [status|navigate <url>|close]" }

func (c *BrowserCommand) Execute(ctx context.Context, cc *CommandContext) (*CommandResult, error) {
    sub := "status"
    if len(cc.Args) > 0 { sub = cc.Args[0] }

```

```

switch sub {
case "status":
    st := c.mgr.Status()
    return &CommandResult{Output: fmt.Sprintf(
        "browser: active=%v\nchromium: %s\nscreenshot_dir:
        %s\nheaded: %v\ncreated_at: %s",
        st.Active, st.ChromiumPath, st.ScreenshotDir,
        st.Headed, st.CreatedAt.Format(time.RFC3339))}, nil
case "navigate":
    if len(cc.Args) < 2 { return nil, fmt.Errorf("/browser
    navigate: url required") }
    url := cc.Args[1]
    s, err := c.mgr.EnsureSession(ctx)
    if err != nil { return nil, err }
    var title string
    if err := s.Run(ctx, chromedp.Navigate(url),
    chromedp.WaitReady("body", chromedp.ByQuery),
    chromedp.Title(&title)); err != nil {
        return nil, err
    }
    return &CommandResult{Output: fmt.Sprintf("navigated:
    %s\ntitle: %s", url, title)}, nil
case "close":
    if err := c.mgr.CloseSession(); err != nil { return nil,
    err }
    return &CommandResult{Output: "browser: closed"}, nil
default:
    return nil, fmt.Errorf("/browser: unknown subcommand %q
    (want status|navigate|close)", sub)
}
}

```

3.6 Per-tool RequiresApproval rationale

- `browser_navigate` → `LevelEdit` — visiting a URL can fetch malicious JS that exploits chromium; this is a real attack vector. Gating behind `ModeAutoEdit+` matches the security posture of `fs_write` (which the user knows mutates state). NOT `LevelRun` because no shell command is exposed.
- `browser_snapshot` → `LevelReadOnly` — pure read; serialises the current DOM. No external side effects. Bypasses approval (F21 §3.6 explicit allowlist).
- `browser_click` → `LevelEdit` — clicking a button can trigger destructive forms (delete, transfer, submit) or navigate to a new URL with new JS. Gated.
- `browser_type` → `LevelEdit` — typing into a field can submit credentials, exfiltrate keystrokes via JS handlers, or fill destructive forms. Gated.
- `browser_screenshot` → `LevelReadOnly` — pure pixel capture; writes to local `tempdir` but no network/DOM side effects. Bypasses approval.
- `browser_close` → `LevelEdit` — terminates the session, dropping any unsaved page state (e.g. half-typed text in a form). Gated so the user can refuse if they're mid-task. Idempotent on re-call.

3.7 New external dependencies

Zero new dependencies. chromedp v0.15.1 + cdproto v0.0.0-20260405000525-47a8ff65b46a are already direct deps in helix_code/go.mod (introduced for the existing internal/tools/browser/package). os/exec is not used (chromedp manages the subprocess). os, sync, sync/atomic, time, errors, fmt, strings, path/filepath, context are stdlib. zap is already direct. T10's verification step asserts `git diff --exit-code go.mod go.sum` is no-op.

4. Data flow

1. Startup (cmd/cli/main.go):

- mgr := browser.NewBrowserManager(browser.NewDefaultChromeDiscovery(), c.logger).
- if err := browser.RegisterAll(c.toolRegistry, mgr); err != nil { log.Fatalf(...) }.
- c.commandRegistry.Register(commands.NewBrowserCommand(mgr)).

2. First browser_navigate(url):

- F21 approval gate consults tool.RequiresApproval() == LevelEdit; in ModeSuggest returns ErrApprovalRequired; in ModeAutoEdit+ allows.
- BrowserNavigateToolV2.Execute(ctx, params) extracts url.
- mgr.EnsureSession(ctx) — atomic load is nil → acquire mu → discover chromium via discovery.Discover() → chromedp.NewExecAllocator(parent, chromedp.NoFirstRun, chromedp.Headless if !opts.Headless==false_implies_headed_else_headless, chromedp.WindowSize(opts.ViewportWidth, opts.ViewportHeight)) → chromedp.NewContext(allocCtx, chromedp.WithLogf(zapInfo)) → mkdir per-session tmpdir → store in current → return.
- session.Run(ctx, chromedp.Navigate(url), chromedp.WaitReady("body", chromedp.ByQuery), chromedp.Title(&title), chromedp.Location(&resolvedURL)).
- Return {url: resolvedURL, title: title}.

3. browser_snapshot(mode):

- mgr.RequireSession() — atomic load; nil → ErrNoActiveSession.
- var content string; var pageURL, pageTitle string.
- mode == "html": session.Run(ctx, chromedp.OuterHTML("html", &content, chromedp.ByQuery), chromedp.Location(&pageURL), chromedp.Title(&pageTitle)).
- mode == "text": session.Run(ctx, chromedp.Text("body", &content, chromedp.NodeVisible, chromedp.ByQuery), chromedp.Location(&pageURL), chromedp.Title(&pageTitle)).
- truncated := false; if len(content) > MaxSnapshotBytes { content = content[:MaxSnapshotBytes]; truncated = true }.
- Return Snapshot{URL: pageURL, Title: pageTitle, Mode: mode, Content: content, Truncated: truncated}.

4. browser_click(selector):

- mgr.RequireSession().

- `session.Run(ctx, chromedp.Click(sel, chromedp.NodeVisible, chromedp.ByQuery), chromedp.Sleep(opts.ClickWaitDuration)).`
 - On selector miss (chromedp returns wrapped `context.DeadlineExceeded` because `NodeVisible` timed out): map to `ErrSelectorNotFound`.
 - 5. `browser_type(selector, text):`
 - `mgr.RequireSession().`
 - `session.Run(ctx, chromedp.SendKeys(sel, text, chromedp.NodeVisible, chromedp.ByQuery)).`
 - 6. `browser_screenshot(full_page):`
 - `mgr.RequireSession().`
 - `var buf []byte.`
 - `full_page == true: session.Run(ctx, chromedp.FullScreenshot(&buf, 90)).`
 - `full_page == false: session.Run(ctx, chromedp.CaptureScreenshot(&buf)).`
 - Verify `len(buf) > 0`; verify PNG magic `0x89 0x50 0x4E 0x47` at offset 0; if absent → fail with descriptive error (real chromium output is PNG; bad bytes mean chromedp returned junk).
 - `path := session.NextScreenshotPath(); os.WriteFile(path, buf, 0600).`
 - Stat the file; assert `Bytes > 1024` (defensive — a 0-byte or 1-byte file is a bug); read width/height by decoding the PNG header (lightweight: `image/png.DecodeConfig` on the first 64 bytes).
 - Return `ScreenshotResult{Path, Bytes, Width, Height}`.
 - 7. `browser_close():`
 - `mgr.CloseSession().` If current is nil → no-op success. Otherwise swap nil + call `prev.Close()` which `s.cancel()`s the chromedp context (terminating chromium) and `os.RemoveAll(screenshotDir).`
 - 8. `/browser status:`
 - Calls `mgr.Status()` which atomically loads current and returns the immutable status snapshot.
-

5. Error handling, anti-bluff hot zone, edge cases

5.1 Error handling

- **Chromium not found** — `discovery.Discover()` returns `ErrChromiumNotFound`; `EnsureSession` propagates; `browser_navigate` fails with a clear error. The CLI does NOT crash; subsequent invocations of any browser tool fail the same way until chromium is installed.
- **Navigation timeout** — `chromedp.Navigate` blocks until `WaitReady("body")` resolves; bounded by `opts.NavigateTimeout = 30 s`. On timeout, `chromedp.Run` returns `context.DeadlineExceeded`; tool wraps as `ErrNavigationTimeout`.
- **Selector not found** — `chromedp.NodeVisible` waits for the selector to become visible; on timeout (default 0 in chromedp = inherits parent ctx; we wrap a `context.WithTimeout(parent, 5s)` for click/type) returns wrapped deadline error → mapped to `ErrSelectorNotFound`.
- **Screenshot empty / non-PNG** — defensive PNG-magic check after `FullScreenshot`. If chromedp returns a non-PNG byte buffer (this would be a chromedp bug, but defensive), tool returns an explicit error rather than writing junk to disk.

- **Close on nil session** — `CloseSession()` is idempotent: if `current.Load()` is nil, returns nil. `browser_close` tool propagates success. (No “you must navigate first” error — the agent should be free to call close defensively.)
- **Concurrent tool calls** — atomic-pointer reads in `RequireSession` are lock-free; `chromedp` itself serialises actions per-context, so concurrent click + snapshot will queue (not race). Tests assert this.

5.2 Anti-bluff hot zone — five critical patterns

Bluff #1: “Navigate succeeded but page didn’t load.” - Pattern: `chromedp.Run(ctx, chromedp.Navigate(url))` returns nil even though the load failed silently (e.g. chromium hit a TLS error and is on `chrome-error://`). The tool reports success; a subsequent snapshot would return empty/error HTML. - Test: integration

`TestBrowser_NavigateThenSnapshot_Real` spins up a real `httptest.Server` serving fixture HTML containing a sentinel string `<p id="fixture">FIXTURE_LOADED_42</p>`; calls `browser_navigate(server.URL)` then `browser_snapshot(mode=html)`; asserts `Snapshot.Content` contains `FIXTURE_LOADED_42`. - Challenge: PHASE-A asserts the same against a Challenge-owned `httptest.Server`.

Bluff #2: “Click succeeded but DOM unchanged.” - Pattern: `chromedp.Click(sel, chromedp.NodeVisible)` matches a no-op element (e.g. a disabled button), or the click handler is async and DOM hasn’t mutated by the time snapshot reads. The tool reports success; the LLM proceeds as if the click did something. - Test: integration

`TestBrowser_ClickMutatesDOM_Real` serves a fixture with `<button id="b" onclick="document.getElementById('out').innerText='CLICKED_42'">go</button><span id="out">UNCLICKED</span>`; navigate → snapshot (assert `UNCLICKED` present) → click #b → snapshot (assert `CLICKED_42` present AND `UNCLICKED` absent — positive byte differential, not just “no error”). - Challenge: PHASE-C identical pattern.

Bluff #3: “Snapshot returned empty HTML.” - Pattern: `chromedp.OuterHTML("html", &content)` runs before `WaitReady("body")` resolves, returning the empty `<html></html>` shell. Tool reports success with empty content. - Test:

`TestBrowser_Snapshot_NotEmpty` asserts `len(Snapshot.Content) > 100` after navigating to fixture (positive evidence — empty would be < 50 bytes for `<html></html>` shell). - Challenge: PHASE-B asserts `len > 100` AND fixture sentinel present.

Bluff #4: “Screenshot file exists but is 0 bytes / not a PNG.” - Pattern:

`chromedp.FullScreenshot(&buf, 90)` returned nil, nil on a closed session; `os.WriteFile(path, buf, 0600)` writes a 0-byte file; tool reports success with the (nonexistent) byte count. - Test: `TestBrowser_Screenshot_PNGMagic_RealFile` reads the first 8 bytes of the returned file and asserts they equal `[]byte{0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A, 0x1A, 0x0A}` (the PNG signature). Asserts `Bytes > 1024` (a real screenshot of even a blank page is multiple KB). - Challenge: PHASE-E identical PNG-magic + size assertions.

Bluff #5: “browser_close didn’t actually terminate chromium.” - Pattern: the tool sets `manager.current = nil` via the swap but never calls `cancel()`, leaving the chromium subprocess running forever (memory leak; persistent listener on a TCP port). Tool reports success; a `pgrep chromium` would still find the PID. - Test:

`TestBrowser_Close_KillsChromium_Real` records the chromium PID via `chromedp.Targets(ctx)` before close; closes; reads the PID’s `/proc/<pid>/status` (Linux); asserts the process is gone (or in zombie state) within 2 s. (Cross-platform fallback: `RequireSession` returns `ErrNoActiveSession` after close — positive functional

evidence.) - Challenge: PHASE-F asserts post-close browser_snapshot fails with ErrNoActiveSession (positive: the manager actually torn down) AND chromedp's chromedp.FromContext(prevCtx). Browser returns an error indicating the connection is closed.

5.3 Edge cases

- **Slow page (> 30 s)** — NavigateTimeout fires; tool returns ErrNavigationTimeout. User can re-call with their own longer ctx.
 - **Cross-origin iframe in snapshot** — chromedp.OuterHTML("html", &c) returns the top-frame HTML; cross-origin iframes are opaque (chromium policy). Acceptable — out of scope per §8.
 - **Headed mode with no display server** — chromedp's exec allocator passes --display-less flags; on Linux without X/Wayland, headed mode silently falls back to headless or fails to spawn. Documented; tests run only headless.
 - **Two concurrent browser_navigate calls** — second call hits the post-Lock double-check, reuses the existing session, navigates within it (both calls observe the second URL after they both return). Not a bug; matches single-tab semantics.
 - **Path traversal in screenshot dir** — paths are constructed via filepath.Join(screenshotDir, fmt.Sprintf("%d.png", n)) with monotonic n; user input never reaches the path. Defensive.
 - **Chromium consumed > 5 MB on a screenshot** — BrowserScreenshotTool checks int64(len(buf)) > opts.ScreenshotMaxBytes and falls back to viewport-only capture (a re-run with chromedp.CaptureScreenshot). If still over, returns ErrScreenshotTooLarge.
 - **Process exit while session active** — cmd/cli/main.go SHOULD defer mgr.CloseSession() in its main loop; tested by sending SIGINT during integration test (assert chromium PID is gone after CLI exit).
-

6. Testing

6.1 Unit tests (mocks ALLOWED)

- types_test.go — pin EnvVarHeadedMode, MaxSnapshotBytes, MaxScreenshotBytes byte-for-byte; assert sentinel errors are distinct + errors.Is-comparable.
- options_test.go — OptionsFromEnv with env unset (Headless=true), with HELIXCODE_BROWSER_HEADED=true (Headless=false), with =TRUE (case-insensitive; Headless=false), with =anything-else (Headless=true).
- manager_test.go — table-driven against a stub ChromeDiscovery that returns a fake binary path AND a stub session-factory. Cases: nil → EnsureSession populates; double EnsureSession returns same pointer; CloseSession swaps to nil + invokes Close; RequireSession after Close returns ErrNoActiveSession; concurrent EnsureSession from N=10 goroutines yields the same pointer.
- session_test.go — primarily tests NextScreenshotPath monotonicity + Close idempotency under sync.Once; full chromedp.Run path is tested under integration (skip-OK if chromium absent).
- navigate_tool_test.go / snapshot_tool_test.go / click_type_tools_test.go / screenshot_tool_test.go / close_tool_test.go — Schema/Validate/Name/RequiresApproval assertions; Execute tests use a stub manager that returns a stub session whose Run records the actions for assertion.

- `browser_command_test.go` — status/navigate/close subcommands with a stub manager.

6.2 Integration tests (NO mocks; //go:build integration; gated on chromium availability)

`tests/integration/browser_test.go` runs ALL tests against a real chromium subprocess managed by `chromedp` + a real `net/http/httptest`. Server serving fixture HTML. Each test starts with:

```
//go:build integration
```

```
func setupBrowserSession(t *testing.T) (*browser.BrowserManager,
    *httptest.Server, func()) {
    // Skip if chromium not on PATH.
    if _, err :=
        browser.NewDefaultChromeDiscovery().Discover(context.Background())
        err != nil {
        t.Skipf("SKIP-OK: #P2-F23 chromium not available: %v",
            err)
    }
    fixture := `<!doctype html><html><head><title>F23-FIXTURE</
        title></head>
        <body>
            <p id="fixture">FIXTURE_LOADED_42</p>
            <button id="b"
                onclick="document.getElementById('out').innerText='CLICKED_42'">go
                button>
                <span id="out">UNCLICKED</span>
                <input id="in" type="text">
        </body></html>`
    srv := httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, _ *http.Request) {
        w.Header().Set("Content-Type", "text/html;
            charset=utf-8")
        _, _ = w.Write([]byte(fixture))
    })))
    mgr :=
        browser.NewBrowserManager(browser.NewDefaultChromeDiscovery(),
            zap.NewNop())
    return mgr, srv, func() { _ = mgr.CloseSession();
        srv.Close() }
}
```

Tests: - `TestBrowser_NavigateThenSnapshot_Real_AssertFixtureSentinel`
 — assert `Snapshot.Content` contains `FIXTURE_LOADED_42` AND `len > 100`. -
`TestBrowser_ClickMutatesDOM_Real` — pre-click snapshot has `UNCLICKED`; click
`#b`; post-click snapshot has `CLICKED_42` (and NOT `UNCLICKED`). -
`TestBrowser_TypeIntoInput_Real` — type “hello” into `#in`; snapshot HTML contains
`value="hello"` OR (more reliable) JS-eval reads back the input value. -

TestBrowser_Screenshot_FileExistsAndPNG_Real — assert returned path exists, Bytes > 1024, first 8 bytes are PNG magic, decoded width/height match the requested viewport. - TestBrowser_Close_RequireFailsAfter_Real — close → call mgr.RequireSession() → assert errors.Is(err, ErrNoActiveSession). - TestBrowser_ConcurrentEnsureSession_SamePointer_Real — fire 10 goroutines into EnsureSession; assert the returned *BrowserSession pointer equality (only one chromium subprocess). - TestBrowser_HeadedMode_OptIn — set env, construct, assert Status().Headed == true. (No display assertion — runs headed flag in BrowserOptions only.)

6.3 Challenge harness — seven phases

challenges/p2-f23-cline-browser-tool/run.sh invokes tests/
integration/cmd/p2f23_challenge/main.go:

1. **PHASE-A: NAVIGATE-AND-SNAPSHOT (always runs; gated on chromium)** — local httptest.Server with sentinel FIXTURE_LOADED_42; browser_navigate(srv.URL) then browser_snapshot(mode=html); assert (i) content contains sentinel, (ii) len(content) > 100, (iii) Snapshot.URL ends with /, (iv) Title equals F23-FIXTURE.
2. **PHASE-B: SNAPSHOT-MODE-TEXT (always runs)** — same fixture; browser_snapshot(mode=text); assert content is a string (not HTML tags) AND contains FIXTURE_LOADED_42.
3. **PHASE-C: CLICK-MUTATES-DOM (always runs)** — pre-click snapshot has UNCLICKED; browser_click("#b"); post-click snapshot has CLICKED_42 (positive byte differential).
4. **PHASE-D: TYPE-INTO-INPUT (always runs)** — browser_type("#in", "HELIX_42"); assert post-snapshot HTML contains either value="HELIX_42" or (via JS-eval helper if available) the input value reads back as HELIX_42.
5. **PHASE-E: SCREENSHOT-PNG-MAGIC (always runs)** — browser_screenshot(full_page=false); assert (i) returned path exists, (ii) os.Stat().Size() > 1024, (iii) first 8 bytes are exactly 89 50 4E 47 0D 0A 1A 0A, (iv) image/png.DecodeConfig succeeds (positive: it's a real PNG, not just magic-bytes-then-junk).
6. **PHASE-F: CLOSE-TEARS-DOWN (always runs)** — browser_close(); subsequent browser_snapshot() returns error errors.Is(err, ErrNoActiveSession). (Positive evidence the manager actually torn down, not just status-flipped.)
7. **PHASE-G: CONCURRENT-SESSION-SHARING (always runs)** — 5 goroutines call EnsureSession simultaneously; assert all 5 receive the same *BrowserSession pointer (proves no per-call chromium spawn).

Output skeleton ends with:

SUMMARY: PHASE-A=4/4 PASS; PHASE-B=2/2 PASS; PHASE-C=3/3 PASS; PHASE-D=2/2
PHASE-E=4/4 PASS; PHASE-F=2/2 PASS; PHASE-G=2/2 PASS

The Challenge MUST exit non-zero on any byte-evidence mismatch. Skip is permitted ONLY when chromium is absent; skip MUST emit SKIP-OK: #P2-F23 chromium not available with the discovery error in the message. Absence-of-error is NEVER acceptable as PASS.

7. Cross-platform

- **Linux** — chromedp's NewExecAllocator finds chromium/google-chrome via the existing ChromeDiscovery interface (internal/tools/browser/discovery.go already implements the search across snap/Flatpak/system installs). httptest.Server uses 127.0.0.1 ephemeral port; chromedp connects locally.
 - **macOS** — ChromeDiscovery already searches /Applications/Google Chrome.app/. . . /Chromium, Brave, Edge. Same code path; same test surface.
 - **Windows** — ChromeDiscovery already searches Program Files/Program Files (x86). Same code path.
 - **No chromium** — discovery.Discover returns ErrChromiumNotFound; integration tests SKIP with SKIP-OK: #P2-F23 chromium not available (per Rule 5 + the no-silent-skips Make target). The Challenge harness similarly skips with explicit log line; CI without chromium reports SKIP-OK and overall exit 0 (skip is not a pass and not a fail).
-

8. Out of scope

- **Multi-tab support** — single-session, single-tab. Multi-tab requires a tab-id parameter on every tool, which doesn't match the cline pattern. v2.
 - **Persistent cookies / sessions across CLI runs** — each Helixcode session starts a fresh chromium subprocess. v2 may add a BrowserUserDataDir option for --user-data-dir=<path>.
 - **Download support** — chromedp.SendDownloadHeaders etc. is a separate surface; out of scope.
 - **PDF rendering** — chromedp.PrintToPDF is a separate tool surface; v2.
 - **Mobile viewport emulation** — chromedp.EmulateViewport(375, 812, chromedp.EmulateScale(2)) etc. is out of scope; users can set HELIXCODE_BROWSER_VIEWPORT in v2.
 - **Browser extension management** — out of scope.
 - **Selenium / Playwright fallback when chromedp fails** — chromedp + chromium is the only supported runtime; alternative engines are v2.
 - **Network interception / request mocking** — out of scope; chromedp supports it via cdproto/network but that's a separate tool surface.
 - **Authenticated browsing (login flows persisted)** — handled implicitly by typing credentials, but no separate "login session" abstraction; v2.
 - **Concurrency beyond single-session sharing** — no parallel multi-session; the manager rejects a second session creation while current != nil.
-

9. Constitutional compliance

- **CONST-035** (anti-bluff): every PASS in F23 carries positive runtime evidence — real chromium subprocess + real httptest.Server + real DOM-mutation byte differential + real PNG-magic verification + real close-then-fail-on-require evidence. The Challenge harness MUST exit non-zero on byte mismatch. Tests use real chromium discovery + real chromedp Run (mocks of chromedp are forbidden in integration tests per Rule 5).
- **CONST-039** (Challenge required): F23 ships with challenges/p2-f23-cline-browser-tool/ (Challenge harness with 7 phases A-G).
- **CONST-042** (no secret leak): full page contents are NEVER logged at INFO level. The browser tools' loggers log only the URL (which is a CALLER input — the agent or user

already knows it), the snapshot byte length, and the screenshot file path. A unit test scans `internal/tools/browser/*.go` for `logger\Info\(.*\b(content|html|page|body|snapshot|outerHTML|innerText)\b` matches and FAILs on any hit (excluding test files). Per-tool descriptions and telemetry NEVER include the user-typed text from `browser_type`. Screenshot files live under user-only-readable `0600` mode in `$XDG_DATA_HOME/helixcode/browser/screenshots/`.

- **CONST-043** (no force push, no auto-push): F23 emits zero `git push` commands. T10's close-out push to four remotes (origin / helixdev / vasic-digital / gitlab) is performed by the human operator with explicit per-push approval per CONST-043; the docs work itself doesn't push.
 - **CONST-033** (host power management): F23 emits no shell commands beyond `chromedp's` chromium-subprocess management (which is a runtime user-space process, not a power-state transition). No `suspend/reboot/halt` commands.
-

10. Open questions resolved

- **Q1 = A** — Browser engine: `chromedp` (already in `helix_code/go.mod` as a direct dep at v0.15.1, with `cdproto v0.0.0-20260405000525-47a8ff65b46a`). Chromium subprocess managed by `chromedp` via `NewExecAllocator + NewContext`. No new external deps.
 - **Q2 = A** — Tool surface: six tools (`browser_navigate`, `browser_snapshot`, `browser_click`, `browser_type`, `browser_screenshot`, `browser_close`); each registered through the F21-extended Tool interface; per-tool `RequiresApproval` mapped per §3.6 (`navigate/click/type/close` → `LevelEdit`; `screenshot` → `LevelReadOnly`).
 - **Q3 = A** — Headless default; `HELIXCODE_BROWSER_HEADED=true` env var (case-insensitive equal to literal `true`) enables headed mode for human debugging. Read once at session-create via `OptionsFromEnv()`; not re-read mid-session.
 - **Q4 = A** — Screenshots written to `$XDG_DATA_HOME/helixcode/browser/screenshots/<session-id>/<n>.png`; tool result returns the absolute file path (NOT base64); files are mode `0600`; per-session `tmpdir` is removed on `CloseSession()`. `n` is monotonic per session via `atomic.Uint64`.
 - **Q5 = A** — `/browser slash` command (`status / navigate <url> / close`) PLUS the six tools as the LLM-facing surface. NO cobra subcommand. Slash + tools share the single `BrowserManager`.
-

11. Non-obvious calls

1. **Coexistence with the existing** `BrowserLaunchTool/BrowserNavigateTool/BrowserScreenshotTool/BrowserCloseTool` in `internal/tools/browser_tools.go`. The existing API takes a `browser_id` per call (multi-browser model from earlier work). F23 lands the cline single-session pattern as a DIFFERENT surface; the existing API stays. T09's registration step picks ONE strategy: (a) register F23 tools under names `browser_navigate/etc.`, overshadowing the existing ones (requires removing/renaming existing ones — riskier for any caller currently depending on them), OR (b) register F23 tools under a `_v2` suffix, leaving both APIs available. Spec defaults to (a) with the existing ones renamed `_legacy` if any test references them. Tests + Challenge use the F23 names.

2. **Atomic-pointer over RWMutex for current.** RequireSession is called on every tool invocation; lock-free read avoids contention under concurrent agent calls. The mutex serialises only the EnsureSession/CloseSession transitions (rare events).
3. **chromedp.WaitReady("body") is mandatory after Navigate.** Without it, OuterHTML reads the empty pre-load <html></html>. Spec §4 step 2 + §5.2 Bluff #1 + #3 pin this.
4. **chromedp.NodeVisible on Click/SendKeys.** Without it, clicks may fire on hidden/zero-pixel elements, producing fake-success. Spec §4 steps 4-5 + §5.2 Bluff #2 pin this.
5. **PNG-magic verification on screenshot bytes BEFORE writing to disk.** Catches the chromedp-returned-empty-buf case at the source rather than discovering it later when a downstream tool tries to decode the file. Spec §5.2 Bluff #4 pin.
6. **sync.Once on BrowserSession.Close().** browser_close followed by process-exit's defer CloseSession() would otherwise call cancel() twice; chromedp tolerates it but the subsequent os.RemoveAll(screenshotDir) would race. sync.Once makes Close idempotent.
7. **Per-session tempdir under \$XDG_DATA_HOME (NOT os.TempDir()).** XDG persists across logins (so the user can find the screenshots after the session); os.TempDir() is /tmp which gets cleaned on reboot. Per-session subdir prevents cross-session collision; close-time RemoveAll keeps disk usage bounded.
8. **Headed mode via env var (NOT a CLI flag).** A flag would force the user to remember it across re-launches; an env var sticks in their shell profile naturally. Read once at session-create — re-reading mid-session would let /browser status lie about the current chromium's actual state.
9. **HELIXCODE_BROWSER_HEADED is case-INsensitive true.** Typos (True, TRUE, yes, 1) are treated as headless (the safe default), NOT as headed. Only the literal lowercase-or-uppercase true enables headed. Tests pin this.
10. **Per-tool RequiresApproval() differs across the six tools.** Reads (snapshot, screenshot) bypass approval; writes (navigate, click, type, close) gate. This is finer-grained than treating “all browser tools” as one approval class — the user benefits from being able to read the page without prompts but being prompted before any mutation.
11. **browser_navigate is LevelEdit, NOT LevelRun.** Even though it spawns a chromium subprocess on first call, the subprocess is owned by chromedp (long-lived; not invoked per-tool). Treating it as LevelRun would force ModeFullAuto to drop it through the F14 sandbox, which doesn't make sense (the chromium binary is itself the sandboxed surface; double-sandboxing breaks chromedp's subprocess management).
12. **browser_close is LevelEdit, NOT LevelReadOnly.** Closing a session drops in-flight page state (half-typed text, mid-load downloads). Gating prevents the agent from “helpfully cleaning up” while the user is mid-task.
13. **F22 auto-commit interaction.** RequiresApproval == LevelEdit for navigate/click/type/close means F22's fireAutoCommit hook will fire after each successful call. That's wrong semantically — these tools don't mutate the working tree. Spec §3.5 of F22 derives MutatedPaths from per-tool param keys; browser_* tools have no path param, so the fallback returns nil, and F22's committer's git status --porcelain check returns “no changes” → Skipped. Net effect: no spurious commits. Verified by integration test in T09.
14. **F21 --approval flag interaction.** In ModeSuggest, browser_navigate returns ErrApprovalRequired immediately without spawning chromium. In ModeAutoEdit, browser_navigate proceeds. Tests cover both directions.
15. **F13 LSP auto-trigger interaction.** F13 fires post-Execute for tools with edited file paths. browser_* tools have no file paths, so F13's path-derivation returns empty and the LSP trigger is a no-op. No interference.
16. **F04 worktree integration.** Since browser tools don't operate on the filesystem (other than the screenshot tempdir), worktree isolation is irrelevant. Each subagent gets its own

BrowserManager if F04 dispatches with isolated registries; otherwise they share. Spec doesn't pin a choice — main.go's wiring decides. T09 default: shared per-CLI-instance, like F22's autocommitter.

17. **Disk usage discipline.** Screenshots accumulate under `$XDG_DATA_HOME` until `CloseSession()` is called. A long-lived agent that takes 100 screenshots without closing accumulates 100 PNGs. Spec §5.3 notes `process-exit's defer CloseSession()` is the safety net. v2 may add per-session quota.
18. `browser_screenshot's decoded width/height comes from image/png.DecodeConfig reading the first 64 bytes (the IHDR chunk).` This is a stdlib call, not a chromedp call, and it's lightweight (no full-image decode). Provides positive evidence that the bytes are a valid PNG, beyond just the magic check.
19. **Context propagation.** Each tool's `Execute(ctx, ...)` parent ctx wraps the chromedp ctx via `chromedp.Run's` internal context handling — chromedp respects ctx cancellation. If the LLM driver cancels the request mid-call (e.g. user hits Ctrl-C), chromium aborts the action cleanly.
20. **Single-source-of-truth for “is a session active”:** `manager.current.Load() != nil`. Both `/browser status` and `RequireSession` use the same atomic load — no two-source-truth drift.